

A 16-byte Asynchronous Gray Code FIFO Memory using Verilog HDL for Real Time Applications

Himanshu
School of VLSI & Embedded System
National Institute Of Technology
Kurukshetra, India
323408201@nitkkr.ac.in

Dr. Chaagan Charan
Department Of Electronics & Communication Engineering
National Institute Of Technology
Kurukshetra, India
chaagan.charan@nitkkr.ac.in

Abstract— Modern CPUs are operating faster than ever because to the quick development of integrated circuits. On hardware, FIFO frequently acts as the buffer for data transmission and reception. In order to transfer data across systems with various time domains quickly and simply, asynchronous FIFOs can solve a variety of clock domain difficulties. However, stable operation of the system requires adjustment of the clock periods. This paper focuses on making the use of two “2 D Flip flop” as synchroniser to avoid the metastability conditions in the design. Universal gates and status flags have been introduced with Gray Code to increase the speed and efficiency of FIFO. It is anticipated that this paper will serve as a resource for IC designers when they are creating interface circuits and helpful in real time applications as it happens on own time and doesn't need scheduling includes emails, company newsletter, chatting on Slack, and Asana and Trello boards. The results have been verified using the software Xilinx Vivado that the proposed asynchronous design is relatively faster, accurate and consumes relatively lesser power while maintaining comparable performance in terms of delays and metastability. The design presented here is for a 16-bytes FIFO memory which can be modified further as per the memory required.

Keywords—synchronizer, clock, asynchronous, status bit

I. INTRODUCTION

With the advancement in the technology of IC design, the scale of circuit design is increasing exponentially. The increasing capabilities of the technology industry is also increasing the complexity in the circuit. So, there is a need to transfer the data between different parts if the system working on the different clock domains. Asynchronous FIFO comes into the picture, which is used to handle such complex data transfer where data bits are written sequentially using a clock domain and are then read sequentially using another clock domain from the same FIFO Memory One of the best techniques for designing an asynchronous FIFO is with the use of Gray code as the pointer address to generate the FIFO Full and FIFO Empty status signals [1].

FIFO are the buffers to store the data. FIFO finds its use in various application of digital systems. A high-speed data like in the optical fibre or ethernet cables need to be accessed by the processor and the memory at different clock cycle. The operating frequencies of various input output devices can be different So, a FIFO is very useful in such applications.

The term FIFO is abbreviated for “First in First Out” i.e., the data values which are written first will be read first. In the asynchronous FIFO, two pointers namely read and write pointer is used which keep the track of addresses where the data is to be written and from where the data is to be read. Write pointer points to the address where the next data value is to be written and the read pointer points to the address from where the next data value is to be read. In the asynchronous FIFO the read operation is done in the read clock cycle whereas the write operation is done in the write clock cycle. The read and write pointers are incremented after each clock cycle to point to next address to allow the read and write of data values respectively. If the read and write operation are done using the same clock signal, then such type of FIFO are known as Synchronous FIFO [2].

II. PROPOSED BLOCK DIAGRAM

The internal structure and components of an asynchronous FIFO is shown below. The write pointer module is connected to FIFO memory via write address and write full flag. The write clock domain is provided with write_clk. The write pointer is further connected to the write synchronizers. [3]

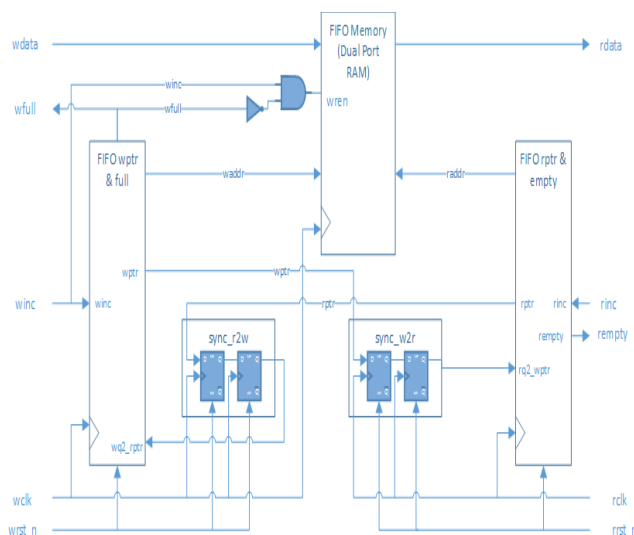


Fig. 1. Asynchronous FIFO block Diagram

The FIFO memory is connected to both read and write pointer. The read pointer operates on the read clock domain. The read clock is connected to read pointer and read synchronizer only, not to FIFO memory. The data is written to the FIFO memory via a signal write_data. It's an 8bit data line used to write into the FIFO memory. [3],[4].

The circuit is also provided with other signals like write_reset, read_reset, write_increment, read_increment which are discussed in details in below section. Different interface signal parameters with their respective size and significance are shown in the table1. This includes write_data and read_data of 8-bits and containing write_full, read_empty and reset of 1-bit. asynchronous FIFOs, which need two separate clocks for read and write operations. Compared to the asynchronous FIFO, the synchronous FIFO is more sophisticated.

TABLE I. DIFFERENT INTERFACE SIGNAL PARAMETERS WITH THEIR RESPE

S.No.	Interface signal	Width	Direction of Signal	Significance
1.	write_data	8 bits	Input signal	The 8-bit data values are sent to the FIFO during write clock cycle
2.	write_full	1 bit	Output signal	Status Flag to show the FIFO is full
3.	read_data	8 bits	Output signal	Read the 8-bit data from the FIFO during read cycle
4.	read_empty	1 bit	Output signal	Status Flag to show FIFO is empty
5.	write_clock	1 bit	Input signal	To provide the write clock to the design
6.	read_clock	1 bit	Input signal	To provide the read clock to the design
7.	write_reset	1 bit	Input signal	To provide the write reset to the design
8.	read_reset	1 bit	Input signal	To reset the read pointer to starting address
9.	write_increment	1 bit	Input signal	To reset the write pointer to starting address
10.	read_increment	1 bit	Input signal	To reset the read pointer to starting address

11.	write_threshold	1 bit	Output signal	Show that write pointer has crossed the 90% size of FIFO Memory
12.	read_threshold	1 bit	Output signal	Show that read pointer has crossed the 90% size of FIFO Memory

A total of 16 Input- Output ports have been used in the paper which includes interface signals their widths and direction of signal with their significance in the paper and 11 all the port pins and signal line used here are mentioned in the table below (See Table1) [4]. This all can be justified by the status bit flag, with the status bit register we can check which pin need to be enable that is write_data or read_pin.

An asynchronous FIFO is a type of Concurrent FIFO which helps to transfer the data from one clock domain to another domain. In the fig.2, the data values are first fed into the FIFO Memory array and then read from the same FIFO memory [5]. This requires two different clocks. The write and read clock are different i.e., read operation and write operation are done at different clock cycles.

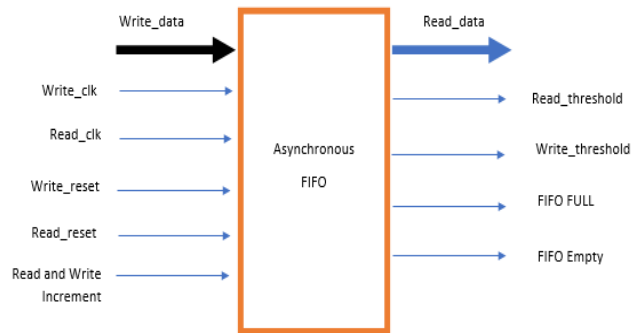


Fig. 2. Black Box view of Asynchronous FIFO

III. MODULES IN ASYNCHRONOUS FIFO

The design of asynchronous FIFO is divided into various parts. Each part each written separately in the form of modules. Each module is then instantiated in the top-level module and connected with proper signals lines. The modules used in our design are -

A. Dual Port RAM Memory Module

Dual port memory allows to perform read and write operation simultaneously. In a dual port RAM, we can perform both read and write operation at a single time [6]. In a single port ram only one operation is performed at a particular clock cycle, be it either read operation or write operation.

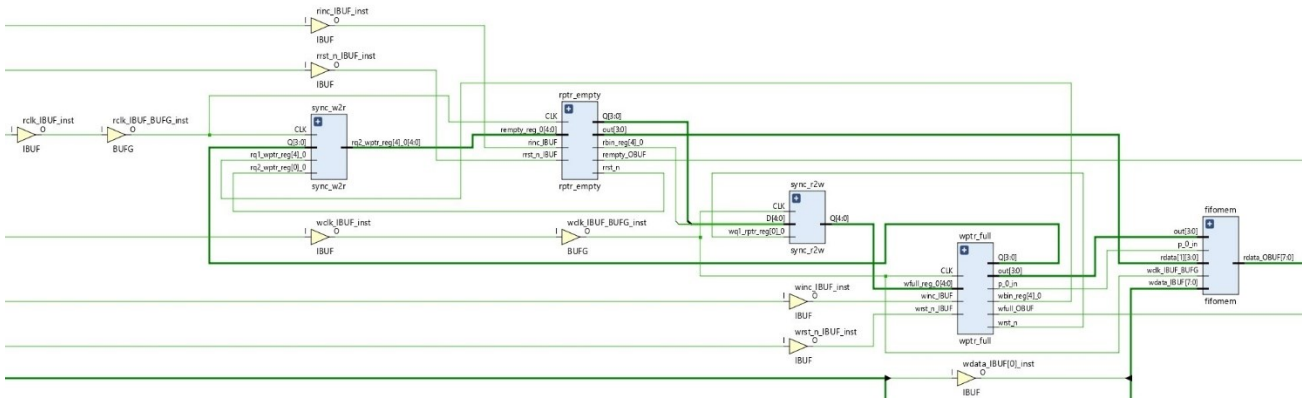


Fig.3. Read Pointer Module RTL Design

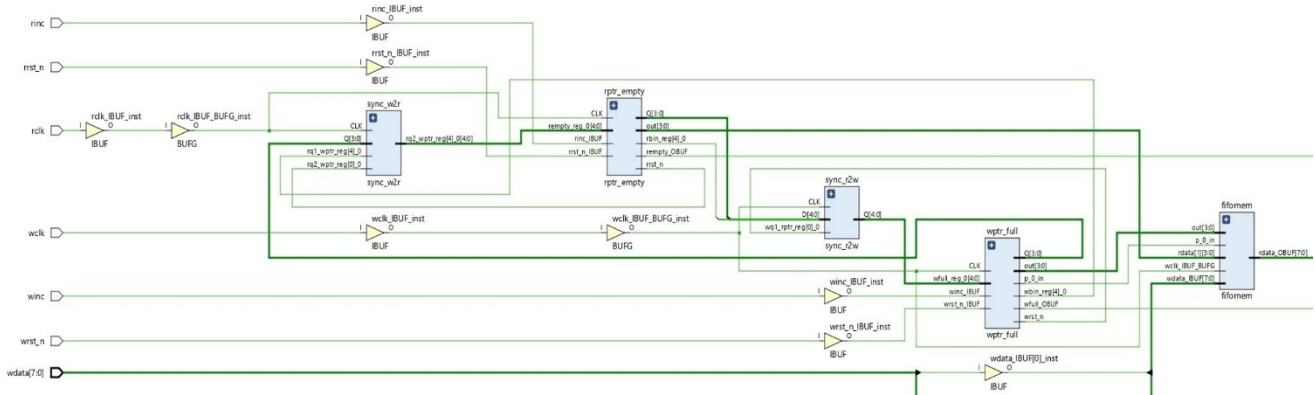


Fig.4. Write Pointer Module RTL Design

B. Read Pointer Module

In Fig.3. the read pointer module contains all the logic of FIFO that are generated in the read clock domain. We have used a Gray code counter in the read pointer too. The 4-bit read address is sent to the FIFO memory to denote the address from where the data values are to be read. The binary address is appended with an extra bit making it a 5-bit pointer [6], [7.]. This 5-bit pointer is then passed to the write synchronizer after converting it into the Gray code.

C. Write Pointer Module

In Fig.4. the write pointer deals with the address where the data is to be written in to the FIFO memory. The write pointer module contains all the logic of FIFO that are generated in the memory to denote the address where the data values are to be written. write clock domain. We have used a Gray code counter in the write pointer [8]. The 4-bit write address is sent to the FIFO memory to denote the address where the data values are to be written.

D. Read & Write Synchronizer

The read synchronizer module used here to synchronize the write pointer into the read clock domain. It receives a 5 bit write pointer which is used by read pointer to generate the conditions of empty FIFO [9],[10],[11].

Module used here to synchronize the write pointer into the read clock domain. It receives a 5 bit write pointer which is used by read pointer to generate the In this read synchronizer there are two D-flip flops synchronized with the read clock domain.

The write synchronizer module used in the paper to synchronize the read pointer into the write clock domain. It receives a 5-bit read pointer which is used by write pointer to generate the conditions of a FULL FIFO. In this write synchronizer also, there are two D-flip flop which are synchronized with the write clock domain [12].

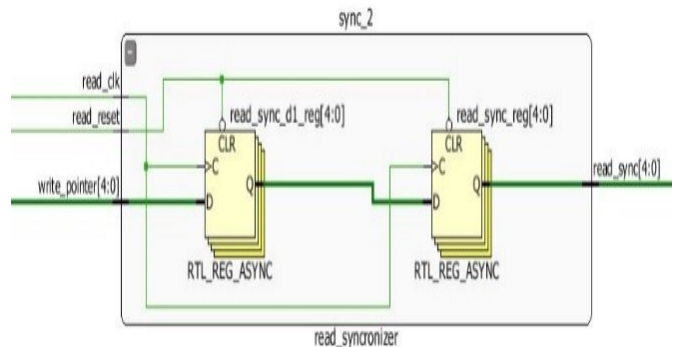


Fig.5. Read Synchronizer RTL Design

The process is writing or storing the data in the FIFO memory up until any flag conditions indicating that writing should stop are raised. The data to be written is provided at the DIN port, the write EN is set high, and the data is written at the subsequent rising edge to complete the write operation.

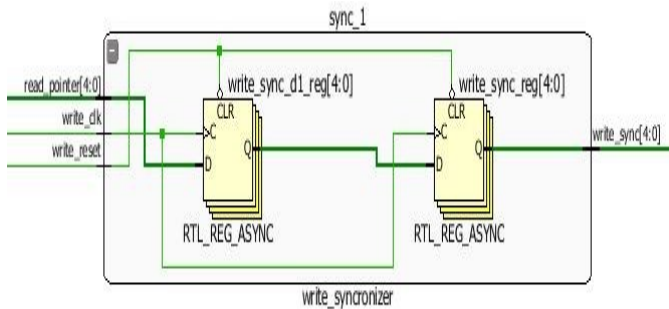


Fig.6. Write Synchronizer RTL Design

IV. DIFFERENT CLOCK DOMAINS

Two different clock domains are used, one for the write operation and other for read operation. These clocks are named as “write_clk” and “read_clk”. The “write_clk” is the clock provided to the write pointer module and the FIFO Memory [12]. The write operation is performed with positive edge of the “write_clk”. The read operation is controlled by the read pointer. The “read_clk” increases the read pointer values (address to be read) on the negative edge of “read_clk” [13]. We have provided a clock of time period 10ns for the write Operation. Frequency of write operation is 100MHz. The read pointer module is provided with a clock of time period 20 ns. Frequency of read operation is 50 MHZ [14],[15].

A. Use of Universal Gates - Nand Gate

There is an operation in the write pointer module which was using one “and” gate and one “not” gate. We have replaced this combination with a NAND gates. It has been replaced with a set of three NAND gates. Further work is being done to replace it further with combination of nor and NAND gates. a.(b)’ was first implemented using “and” gates but the same operation can be made with the help of three NAND gates. The benefit of using NAND gates is that the logical effort of NAND gates is less resulting in charging and discharging of the capacitors fast. Thus, making it more efficient.

B. Use of Status Flag –

We have introduced a new status flag, named as threshold flag. This status flag is provided on both modules read and write pointer_module resulting in “read_threshold” and “write_threshold” respectively.

a) *Read_threshold*: Read threshold is an output signal. Raed threshold staus flag will tell them that 10% of the memory has been read. It can also be set to 90%. This status flag will set to logic high when the read operation has reached to 10% or 90%(can be set by the user).

b) *Write_threshold*: write_threshold is an output signal. Raed threshold staus flag will tell them that 10% of the memory has been read. It can also be set to 90%. This status flag will set to logic high when the read operation has reached to 10% or 90%(can be set by the user).

Benefit of Threshold Status Flag. The threshold flag provides information related to the FIFO near to full or near to empty. We can know how much it has been filled or how much it is empty.

V. RESULT ANALYSIS

Modern CPUs are operating faster than ever because to the quick development of integrated circuits. These days, FIFO frequently acts as the hardware's buffer for data transmission and reception. Through a comparative investigation, this research focuses on the unique qualities of synchronous and asynchronous FIFO. While asynchronous FIFOs can solve a variety of issues across clock domains, including the problem of mismatched read and write speeds, they are generally only used as data caches to transfer data between systems with different clock domains. However, stable operation of asynchronous FIFOs requires adjusting the clock periods.

a) *Simulation Waveforms*: : The Simulation of the design made using Verilog HDL has been done in the Xilinx Vivado 2016.2 software. This shows read and write Operation on different clock domain. It shows that 16 8-bit data are written in the FIFO using write_clk domain and the same data is being read from the FIFO using read_clk domain.

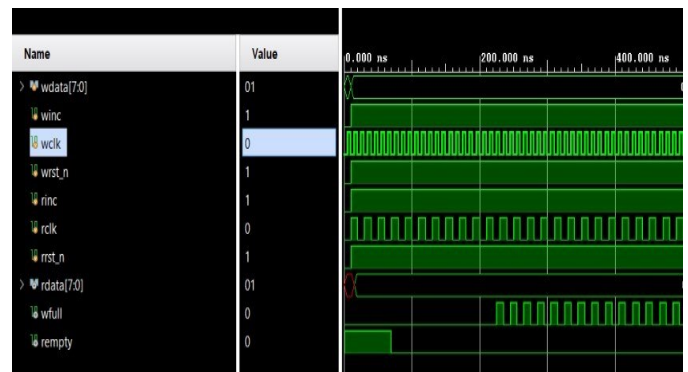


Fig.7. Behavioural Simulation of Asynchronous Gray Code FIFO

The read_empty and write_empty is showing the correct output. These signals are logic high when the FIFO is empty and full respectively. Moreover, we can see the threshold status flags in the same figure. We have set the threshold to 10% for write operation and 90% for read operation.

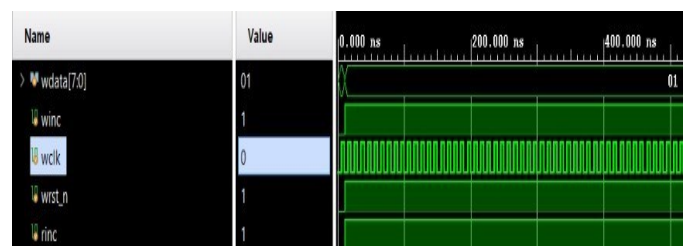


Fig.8. Write Port RAM Memory Simulation

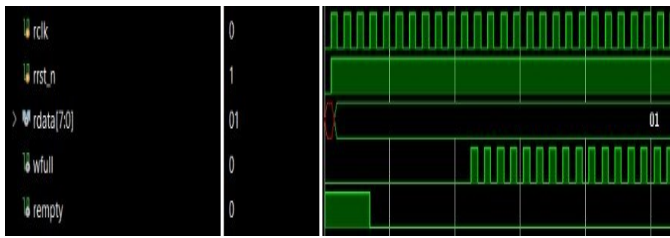


Fig.9. Read Port RAM Memory Simulation

This shows the read and write address, data and full and empty indication with the high and low status bit at a particular time. The simulation results of FIFO memory. It shows that it is storing the data values during the write operation. The addresses where the data is being written is shown.

VI. CONCLUSION & FUTURE SCOPE

Successful implementation of Asynchronous FIFO in Verilog represents a significant achievement in digital communication design. The “AND” and “NOT” gate combination can be replaced with three “NAND” gate combination which results in decreasing logical effort of the circuit. I also noticed that the design was lagging in showing some extra status flag. two more extra flag bits namely – write_threshold and read_threshold. Using these bits as status flags will help to effectively read and write data at the edges i.e., full and empty conditions.

In terms of future scope, enhancements and optimizations can be explored to further refine the Asynchronous FIFO. In Future, many more modifications can be done in the areas of: Accuracy, Efficiency, Speed, Frequency of asynchronous FIFO.

Sectors in which real time applications of Asynchronous FIFO occurs: Sending e-mails, turn an informational meeting to a video call in real time, Collecting real time data from API Keys, Screen casting, blogging, Flipgrid of videos.

REFERENCES

- [1] E. Xie and J. Zhou, "Analysis and Comparison of Asynchronous FIFO and Synchronous FIFO," 2023 IEEE 2nd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA), Changchun, China, 2023, pp. 260-264
- [2] Z. Hao, L. Liu and B. Tian, "The Principle and Applications of Asynchronous FIFO," 2023 IEEE 2nd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA), Changchun, China, 2023, pp. 277-279
- [3] Shivali and M. Khosla, "Coverage of Meta-Stability Using Formal Verification in Asynchronous Gray Code FIFO," 2022 2nd International Conference on Intelligent Technologies (CONIT), Hubli, India, 2022, pp. 1-8
- [4] S. Abdel-Hafeez and M. Q. Quwaider, "A One-Cycle Asynchronous FIFO Queue Buffer Circuit," 2020 11th International Conference on Information and Communication Systems (ICICS), Irbid, Jordan, 2020, pp. 388-393.
- [5] T. -T. Nguyen and X. -T. Tran, "A novel asynchronous first-in-first-out adapting to multi-synchronous network-on-chips," 2014 International Conference on Advanced Technologies for Communications (ATC 2014), Hanoi, Vietnam, 2014.
- [6] Xin Wang, T. Ahonen and J. Nurmi, "A synthesizable RTL design of asynchronous FIFO," 2004 International Symposium on System-on-Chip, 2004. Proceedings., Tampere, Finland, 2004.
- [7] CMOS Digital Integrated Circuits, Sung-Mo-Kang Yusuf Leblebici, Third Edition, 2017.
- [8] Verilog HDL – A guide to Digital Design and Synthesis, Samir Palnitkar, Second Edition, 2003.
- [9] Asynchronous FIFO architecture, functions and applications, Texas instruments, SCAA042A November 1999.
- [10] Design, Simulation and Realization of a Parametrizable, Configurable and Modular Asynchronous FIFO, Haytham Ashour Mentor Graphics, Emulation Division Cairo, Egypt, 2012.
- [11] Coverage of Meta-Stability Using Formal Verification in Asynchronous Gray Code FIFO, Shivali Mamta Khosla, Dr. B.R. Ambedkar National Institute of Technology Jalandhar, Punjab, 2014
- [12] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, Digital Integrated Circuits, A Design Perspective, Prentice Hall, Upper Saddle River, NJ, 2012.
- [13] Clifford E. Cummings, "Simulation and Synthesis Techniques for Asynchronous FIFO Design," SNUG Conference, San Jose, CA, 2002.
- [14] J. T. Yantchev, C. G. Huang, M. B. Josephs and I. M. Nedelchev, "Low-latency asynchronous FIFO buffers," Proceedings Second Working Conference on Asynchronous Design Methodologies, London, UK, 2005.
- [15] Yanjun Zhang, Chunli Yi, Jinqi Wang and Jinye Zhang, "Asynchronous FIFO implementation using FPGA," Proceedings of 2011 International Conference on Electronics and Optoelectronics, Dalian, 2011.